## Cérémonie de définition

La cérémonie de définition transforme le besoin compris en spécifications exploitables par toute l'équipe. Elle sert de pont entre l'élicitation et la planification : on y modélise, valide, mesure les impacts et confirme que ce qui sera planifié repose sur une compréhension partagée, traçable et suffisamment précise.

> **Note terminologique :** cette page décrit la cérémonie de définition dans son contexte opérationnel. Pour les définitions de référence des termes comme **DÉFINIR**, **spécification IF... THEN...**, **spécification atomique**, **SPECIFY**, **DoR**, **DoD**, **validation**, **vérification**, **rôles** et **RACI**, voir le Lexique / Terminologies et la page SPÉCIFICATION. Les anciens termes comme critères d'acceptation, cas, scénarios, Gherkin ou BDD convergent vers une seule nomenclature : la spécification IF... THEN.... Les contributions indiquées ici sont propres à cette cérémonie et ne remplacent pas la matrice RACI.

### 🎯 Objectif

L'atelier de définition transforme les besoins élicités en **spécifications atomiques** — complètes, testables, modélisées et approuvées. C'est ici que le besoin compris devient un besoin **documenté, modélisé, validé et prêt à être réalisé**.

> On ne peut pas bien implémenter ce qu'on n'a pas bien défini. Et on ne peut pas bien définir ce qu'on n'a pas bien élicité.

### 🧠 Travail DEV pendant DÉFINIR : modéliser avant d'implémenter

Pendant l'atelier de définition, **l'équipe de développement n'est pas en mode attente** jusqu'à ce que des stories soient prêtes à implémenter. Au contraire, il y a déjà un travail important et actif côté DEV :

- **Modéliser** le domaine, les flux, les états et les comportements attendus.
- **Challenger** la faisabilité technique des spécifications.
- **Identifier** les impacts systèmes, dépendances et contrats d'intégration.
- **Clarifier** les options de solution possibles.
- **Détecter** les risques techniques, la dette existante et les zones d'incertitude.
- **Contribuer** à rendre les spécifications atomiques testables, cohérentes et implémentables.

> Les développeurs ne sont pas en attente de stories à coder : ils participent activement à définir correctement ce qui devra être construit.

### ⚙️ Ce travail compte dans la capacité

Ce travail de modélisation et d'analyse technique **consomme de la capacité DEV réelle**, même s'il ne produit pas encore directement du code livré en production. Pour les équipes qui travaillent en sprint :

- Ce travail **devrait apparaître dans le sprint** et être comptabilisé dans la vélocité.
- Il reflète la **vraie capacité utilisée** par l'équipe.
- Il évite la pression de « tourner les coins ronds » en créant des stories prématurées juste pour nourrir les développeurs.

> **Il y a du travail dès le jour 1.** Même quand les FEATURE ne sont pas encore prêtes à être découpées en STORY, le travail de définition — modélisation, analyse d'impact, faisabilité, prototypage — ne manque pas.

### 🔄 Concordance 100 % entre modèle, diagrammes et spécifications

Les diagrammes, les modèles et les flux de l'application doivent concorder à **100 %** avec les spécifications — et vice versa.

- Si un diagramme de séquence montre un flux, la spécification textuelle doit décrire le même flux.
- Si une spécification décrit une transition d'état, le modèle doit la représenter.
- Si une spécification IF... THEN... décrit un comportement attendu, le diagramme ou le modèle ne doit pas raconter autre chose.

> Le modèle est la spécification visuelle. La spécification est le modèle textuel. Les deux doivent raconter exactement la même histoire.

### 👥 Participants recommandés

| Rôle | Contribution dans l'atelier de définition |
|---|---|
| **Product Owner / Product Manager** | Valide le scope, la valeur et l'alignement avec le besoin élicité. |

| | |
|---|---|
| **Développeur / Architecte** | Modélise les flux, valide la faisabilité, identifie les impacts et les options techniques. |
| **QA / Qualité** | Utilise les spécifications IF... THEN... comme source directe de validation, explore les cas limites, les niveaux de validation et les risques de régression, sans recréer une couche séparée de critères ou scénarios. |
| **Support / CARE** | Apporte le contexte terrain, les irritants récurrents et valide que la spécification est compréhensible pour le diagnostic. |
| **Utilisateur / Spécialiste** | Confirme que la spécification répond toujours au vrai besoin — directement ou indirectement. |

**Principe :** tout le monde utilise la **même spécification**. Il ne doit pas y avoir une documentation PO, une documentation dev et une documentation QA. Une seule source de vérité.

### 📥 Inputs / Prérequis

Avant de commencer l'atelier de définition, on devrait avoir en main :

| Prérequis | Pourquoi c'est nécessaire |
|---|---|
| **Outputs de la cérémonie d'élicitation** | Besoin compris, reformulé et documenté avant de définir la solution. |
| **FEATURE existante dans Jira** | Rattacher le travail à l'EPIC pertinente et préserver la traçabilité. |
| **Signaux, métriques et RCA collectés** | Ancrer la définition dans des faits observés, pas seulement dans des opinions. |
| **Décision produit explicite** | Confirmer que le sujet doit poursuivre vers DÉFINIR. |
| **Contexte produit** | Comprendre les utilisateurs concernés, la valeur attendue et les contraintes connues. |
| **Décisions passées liées au même sujet** | Éviter de rouvrir des décisions fermées sans nouveaux signaux ou justification claire. |

### 📤 Outputs / Livrables attendus

À la fin de l'atelier, on doit pouvoir répondre à la question : **est-ce qu'on sait exactement ce qui doit être construit, testé et livré ?**

- Des **spécifications atomiques** complètes, testables et non ambiguës.
- Des **modèles et diagrammes** — flux, séquences, machines à états — concordant à 100 % avec les spécifications textuelles.
- Des **spécifications IF... THEN...** mesurables, observables et reliées à des exemples, sans couche séparée de critères d'acceptation.
- L'**analyse d'impact** systémique : modules, équipes, APIs, dépendances et contrats touchés.
- Les **cas limites** explorés : erreurs, pertes réseau, annulations, états concurrents, comportements dégradés.
- Les **risques et la dette technique** identifiés et rendus visibles.
- Les **signaux et métriques produits pendant la définition** : risques quantifiés, dette identifiée, complexité estimée, hypothèses techniques à valider et seuils à surveiller.
- Une **estimation initiale** au niveau EPIC/FEATURE pour soutenir la planification.
- Le **SPECIFY** validé et approuvé avant de passer à PLANIFIER.

### ⏱️ Durée, format et sous-phases recommandées

Comme pour l'élicitation, l'atelier peut être réalisé en **deux sous-phases** :

1. **Modélisation technique initiale** — portée principalement par les développeurs et l'architecte. On modélise les flux, les états, les interactions et on identifie les impacts techniques et les options de solution.
2. **Validation croisée avec les « amigos »** — avec le PO, QA, support et les parties prenantes pertinentes. On valide que les spécifications sont complètes, testables, cohérentes avec le besoin et alignées avec la vision produit.

**Durées indicatives :**

- **Bug simple ou correctif ciblé :** 30 à 60 minutes — le modèle est souvent déjà connu, il suffit de formaliser.
- **FEATURE simple :** 1 à 2 heures pour modéliser, spécifier les critères et valider les impacts.
- **FEATURE complexe ou sujet transversal :** plusieurs séances peuvent être nécessaires. C'est normal.

**Une FEATURE peut rester longtemps en DÉFINI.** C'est attendu quand la modélisation est complexe, quand il y a des options techniques à comparer, des prototypes à valider ou des

dépendances inter-équipes à coordonner. Le travail de modélisation et de formalisation consomme de la capacité réelle et doit être reconnu comme tel.

### 🔄 Déroulement suggéré

1. **Rappeler le contexte** — présenter le besoin élicité, les signaux, les métriques collectées, la FEATURE et la décision de poursuivre.
2. **Modéliser** — construire ou enrichir les diagrammes de flux, machines à états, séquences et modèles de domaine.
3. **Spécifier en IF... THEN...** — formuler les comportements attendus sous forme conditionnelle universelle : **IF** contexte, condition, entrée, état ou événement, **THEN** résultat, comportement, contrainte, effet ou signal observable.
4. **Explorer les cas limites** — que se passe-t-il si le réseau tombe, si l'utilisateur annule, si deux événements arrivent en même temps, si une dépendance est indisponible ?
5. **Analyser les impacts** — identifier les modules, produits, équipes, contrats d'API et processus affectés.
6. **Identifier les risques et la dette** — documenter ce qui pourrait mal tourner et quelle dette technique est créée ou aggravée.
7. **Valider la concordance** — vérifier que modèles, diagrammes et spécifications racontent exactement la même histoire.
8. **Estimer au niveau EPIC/FEATURE** — produire une estimation de haut niveau pour soutenir la roadmap.
9. **Approuver le SPECIFY** — valider avec les parties prenantes, incluant l'utilisateur ou le spécialiste lorsque nécessaire.

1. **Rappeler le contexte** — présenter le besoin élicité, les signaux, les métriques collectées, la FEATURE et la décision de poursuivre.
2. **Modéliser** — construire ou enrichir les diagrammes de flux, machines à états, séquences et modèles de domaine.

### 🔁 Boucle de rétroaction vers ÉLICITER

DÉFINIR ne reçoit pas seulement l'information d'ÉLICITER : la phase doit aussi **challenger ÉLICITER**. Si, en définissant, l'équipe découvre que le besoin est incomplet, contradictoire, trop large, non mesurable ou déconnecté de la réalité utilisateur, l'information doit **retourner à ÉLICITER**.

- Les incohérences deviennent des signaux à clarifier.
- Les métriques observées peuvent remettre en question le besoin initial.

- Les cas limites découverts peuvent enrichir l'élicitation.

> Si en définissant on réalise que le besoin n'est pas clair, ce n'est pas un échec : c'est le système qui fonctionne. On retourne à ÉLICITER, on clarifie, et on revient.

### ➡️ Quand passer à PLANIFIER ?

On peut passer à PLANIFIER lorsque :

- Les spécifications atomiques sont complètes, comprises et documentées.
- Les spécifications IF... THEN... sont testables, mesurables et non ambiguës.
- Les modèles et diagrammes concordent à 100 % avec les spécifications.
- Les impacts systèmes, dépendances et risques sont analysés.
- La dette technique est identifiée et rendue visible.
- La traçabilité besoin → spécification IF... THEN... → validations automatisées ou preuves est assurée.
- Le SPECIFY est validé et approuvé.

> **Test simple :** est-ce qu'une IA pourrait générer le code, les validations et les preuves à partir des spécifications IF... THEN... seules, sans poser de questions majeures ? Si oui, on est prêt pour PLANIFIER.

### ⚠️ Pièges courants à éviter

| Piège | Pourquoi c'est un problème | Comment l'éviter |
|---|---|---|
| Spécifier en silo | Chaque rôle crée sa propre interprétation. | Utiliser une seule documentation partagée. |
| Spécifications vagues | Impossible de tester ou d'automatiser correctement. | Formuler chaque spécification en IF... THEN..., avec un résultat observable, mesurable et des exemples lorsque nécessaire. |
| Modèle ≠ spécification | Le texte et les diagrammes racontent deux histoires différentes. | Valider la concordance 100 % modèle ↔ texte ↔ spécifications. |

| | | |
|---|---|---|
| Ignorer les cas limites | Les défauts apparaissent souvent dans les scénarios non spécifiés. | Explorer systématiquement erreurs, états limites et comportements dégradés. |
| Coder sans SPECIFY approuvé | Le travail démarre avant que la cible soit claire. | Exiger une validation de DÉFINIR avant PLANIFIER. |
| Considérer les DEV en attente | L'équipe perd du temps pendant que la modélisation pourrait avancer. | Les DEV modélisent activement — ce n'est pas du temps mort. |

### 📝 Historique et traces des sessions

Les notes réelles des ateliers de définition peuvent être conservées comme **pages enfants** sous cette cérémonie. Cela permet de garder une trace des décisions, hypothèses, modèles discutés, risques découverts et retours vers ÉLICITER ou vers PLANIFIER.

**Gabarit recommandé pour une session :**

- **Date et contexte** — FEATURE, EPIC, produit, version ou initiative concernée.
- **Participants** — rôles présents et décisions attendues.
- **Besoin élicité de départ** — lien vers la demande, les signaux, métriques ou RCA associés.
- **Modèles et diagrammes discutés** — flux, états, séquences, dépendances et captures pertinentes.
- **Spécifications IF... THEN... produites ou ajustées** — comportements validés, cas limites et preuves attendues.
- **Impacts, risques et dette identifiés** — modules, équipes, APIs, contrats, architecture ou processus touchés.
- **Actions et décisions** — propriétaire, échéance, statut et lien Jira/Confluence lorsque disponible.
- **Retours vers ÉLICITER** — besoins à clarifier, hypothèses à valider ou signaux à retourner.
- **Conclusion** — prêt pour PLANIFIER, à compléter, bloqué ou à retourner en ÉLICITER.

### 📚 Références

- Comment définir — méthode complète de référence pour la phase DÉFINIR.
- Cérémonie d'élicitation — phase précédente qui fournit le besoin, les signaux et les métriques initiales.

- Cérémonie de planification — phase suivante qui transforme les spécifications approuvées en plan d'exécution.
- SDLC officiel OPAL-RT — vue d'ensemble des phases ÉLICITER, DÉFINIR, PLANIFIER, IMPLÉMENTER, DIFFUSER et CONFIRMER.
- Contrat qualité officiel OPAL-RT — exigences minimales de qualité et de traçabilité.
- RACI officiel OPAL-RT — rôles, responsabilités et posture qualité.
- Qualimétrie officielle OPAL-RT — présentation institutionnelle de la mesure qualité.
- Lexique / Terminologies — définitions communes des termes utilisés dans cette cérémonie.